Claude Code 外掛生態系與可配置元件完整解析

Claude Code 已建構起一套以檔案系統為核心、Git 為分發管道的去中心化外掛生態系。 不同於 npm 或 pip 的中央化套件管理器，Claude Code 的 Plugin Marketplace 本質上是一個「以 Git 倉庫為目錄的外掛市集」——任何人都能建立自己的市集，而使用者透過 `/plugin marketplace add` 和 `/plugin install` 兩步驟完成安裝。Anthropic 官方市集 `anthropics/claude-plugins-official` （**9,400+ stars**） github 在啟動時自動可用。截至 2026 年 3 月，整個生態系已涵蓋 **9,000+ 外掛**、 Firecrawl SkillsMP 索引了 **50 萬+技能**， SkillsMP 而 Agent Skills 已被 Anthropic 發布為跨平台開放標準， Anthropic Anthropic 支援 Claude Code、OpenAI Codex CLI、GitHub Copilot、Cursor 及 Gemini CLI 等主流工具。 Unite.AI Microsoft Community Hub

---

外掛市集的運作機制與架構

市集即 Git 倉庫

Claude Code 的市集是一個包含 `.claude-plugin/marketplace.json` 的 Git 倉庫，定義了市集名稱、擁有者及外掛清單。 Claude 每個外掛可以指向相對路徑、GitHub 倉庫（支援 `ref` 和 `sha` 鎖定版本）、npm 套件或任意 Git URL。 Claude

```
{
  "name": "my-marketplace",
  "owner": { "name": "My Team", "email": "team@example.com" },
  "plugins": [
    { "name": "security-suite", "source": "./plugins/security-suite" },
    { "name": "testing-tools", "source": "https://github.com/team/testing-tools"
  ]
}
```

安裝流程為兩步驟：先用 **`/plugin marketplace add owner/repo`** 新增市集（類似新增 App Store），再用 **`/plugin install plugin-name@marketplace-name`** 安裝個別外掛。官方市集 `anthropics/claude-plugins-official` 預設已掛載，使用者可直接透過 `/plugin > Discover` 分頁瀏覽。 Claude github

外掛的完整結構

一個外掛本質上是一個目錄，封裝了所有可配置元件： Angelo Lima Claude

```
plugin-name/
├── .claude-plugin/
│   └── plugin.json          # 外掛元資料（必要）
├── skills/                  # 技能定義（選配）
│   └── skill-name/
│       └── SKILL.md
├── commands/                # 斜線指令（選配）
├── agents/                  # 子代理定義（選配）
├── hooks/                   # 生命週期鉤子（選配）
├── .mcp.json               # MCP 伺服器設定（選配）
└── README.md
```

github

安裝作用域與選擇性安裝

安裝外掛時使用者選擇四種作用域之一：**User scope**（全域個人）、**Project scope**（寫入 `.claude/settings.json` ，團隊共享）、**Local scope**（僅限本機本專案）、**Managed scope**（管理員部署，不可修改）。 Claude Claude

選擇性安裝是架構天生的優勢。 由於每個技能、指令、代理都是獨立的檔案或目錄，使用者可以只複製需要的元件。例如從一個含有 10 個技能的外掛倉庫中只安裝 2 個——直接將對應的 `skills/skill-name/` 目錄複製到 `.claude/skills/` 即可。第三方工具如 CCPM 和 `@tech-leads-club/agent-skills` 更進一步提供了 CLI 指令：

```
npx @tech-leads-club/agent-skills install -s aws-advisor coding-guidelines
```

GitHub

安全機制方面，Claude Code 在安裝外掛時會**將檔案複製到本地快取** `~/.claude/plugins/cache` ，外掛無法引用目錄外的檔案（ `../` 路徑無效），確保沙箱隔離。 Claude Claude

---

Skills：以 SKILL.md 為核心的漸進式能力封裝

技能是什麼

Skills 是 Claude Code 最重要的擴充機制，每個技能是一個自包含的目錄，核心是 **SKILL.md** 檔案—— anthropic 包含 YAML frontmatter 元資料和 Markdown 指令。 Anthropic +3 技能代表特定領域的專業能力， GitHub 例如「PDF 處理」、「安全審計」、「部署工作流」。 Anthropic

SKILL.md 結構

```
---
name: code-review
description: >-
  深度程式碼審查專家。在使用者修改程式碼後自動觸發，
  或在使用者要求 review 時使用。
context: fork
disable-model-invocation: false
---

# Code Review

## 審查流程
1. 分析程式碼結構與設計模式
2. 檢查安全漏洞
3. 評估效能影響
4. 提供具體改善建議

## 參考資料
- 詳細審查標準見 `references/standards.md`
- 邊界案例見 `references/edge-cases.md`
```

必要欄位： `name` （最多 64 字元，小寫加連字號）和 `description` （最多 1024 字元，必須描述何時使用）。** Anthropic +2 選配欄位**包括 `context: fork` （在子代理中隔離執行）、 `disable-model-invocation: true` （僅限使用者手動觸發）、 `user-invocable: false` （僅 Claude 自動觸發的背景知識）、 `model` （覆寫模型）及 `allowed-tools` （限制可用工具）。 Claude

三層漸進式載入架構

這是 Skills 最精妙的設計——**安裝再多技能也不會拖慢回應速度**： claude

層級	載入時機	Token 成本	內容
Level 1：元資料	啟動時永遠載入	~100 tokens/技能	僅 `name` + `description`
Level 2：完整指令	Claude 判斷相關時	<5,000 tokens	SKILL.md 完整內容
Level 3：捆綁資源	按需載入	無上限	`scripts/` 、 `references/` 、 `assets/` 中的檔案

技能選擇完全透過 **LLM 推理**完成——沒有算法路由、意圖分類或嵌入匹配。 Lee Hanchung Claude 讀取所有可用技能的 `name` 和 `description` ，然後用語言理解能力判斷哪個技能適用。 Anthropic +3 Anthropic 官方建議將 description 寫得「積極主動」，因為 Claude 傾向於**不夠頻繁地觸發技能**。 GitHub

社群實際案例

- **feature-dev**（官方外掛，**89,000+ 安裝**）：包含 `code-explorer` 、 `code-architect` 、 `code-reviewer` 三個代理，引導式功能開發 TurboDocx
- **plugin-dev**（官方）：用於開發技能的「元技能」，附帶完整的技能建構指南
- **frontend-design**（官方）：自動偵測前端開發任務並注入設計指引 Claude Code Plugins GitHub
- **obra/superpowers**：社群最受歡迎的技能框架，提供子代理驅動的開發方法論 GitHub
- **PDF/DOCX/PPTX 處理**（Anthropic skills 倉庫）：檔案格式轉換與處理技能 claude GitHub

---

Hooks：事件驅動的確定性控制層

鉤子解決了什麼問題

Hooks 提供了 **LLM 行為的確定性保障**。不像依賴 Claude 自主判斷的技能，鉤子保證「某些事情一定會發生」—— Eesel AI 例如每次編輯 TypeScript 檔案後一定執行 Prettier 格式化， Builder.io 或者一定阻擋包含 `rm -rf` 的指令。 Anthropic

生命週期事件完整清單

鉤子掛載在 Claude Code 的執行生命週期上，目前支援 **18+ 個事件**：

事件	觸發時機	支援 Matcher
PreToolUse	工具執行前（可阻擋）	✅
PostToolUse	工具成功執行後	✅
PostToolUseFailure	工具執行失敗後	✅
UserPromptSubmit	使用者送出提示時	❌
SessionStart	新/恢復工作階段	❌

SessionEnd	工作階段結束	❌
Stop	Claude 完成回應	❌
SubagentStart/Stop	子代理啟動/完成	❌
PreCompact	壓縮對話前	❌
Notification	通知觸發	❌
PermissionRequest	權限請求	✅
WorktreeCreate/Remove	Git worktree 生命週期	❌

配置格式

鉤子在 `settings.json` 中定義， Anthropic +2 支援四種處理器類型：**command**（shell 指令）、**http**（HTTP POST）、**prompt**（Claude 模型評估）、**agent**（代理判斷）。 Anthropic

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/block-dangerous.sh",
        "timeout": 60
      }]
    }],
    "PostToolUse": [{
      "matcher": "Edit|MultiEdit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.file_path' | { read f; [[ $f == *.ts ]] &&
      }]
    }]
  }
}
```

Matcher 支援精確匹配（`"Write"`）、萬用字元（`"*"` 匹配所有工具）及管道分隔多重匹配（`"Edit|MultiEdit|Write"`）。 Anthropic

控制流與輸入/輸出

鉤子透過 **stdin** 接收 JSON 輸入（包含 `session_id`、`tool_name`、`tool_input` 等），透過**退**

出碼控制行為： `exit 0` 放行、 **`exit 2` 阻擋**（stderr 回饋給 Claude）、其他退出碼為非阻擋錯誤。 Anthropic Claude PreToolUse 鉤子還能透過 stdout JSON 的 `updatedInput` 欄位**修改工具輸入**。 ClaudeLog

社群熱門應用

- **GitButler 整合**：在 PreToolUse/PostToolUse/Stop 三個事件上掛載 `but claude` 指令，實現自動分支管理 GitButler Docs
- **hookify 外掛**（官方）：分析對話歷史，自動生成防止不良行為的鉤子 GitHub
- **ralph-loop**（官方）：Stop 事件鉤子，讓 Claude 能自主進行多小時的編碼工作 Firecrawl
- **PR 前置測試閘門**：在 `mcp__github__create_pull_request` 上掛載 PreToolUse 鉤子，確保測試通過才能建立 PR Steve Kinney
- **桌面通知**：Notification 事件觸發 macOS `osascript` 或 Linux `notify-send` Anthropic

---

MCP 伺服器：連接外部世界的開放協議

配置方式

MCP（Model Context Protocol）是 Anthropic 主導的開放標準，讓 Claude Code 連接到外部工具和資料源。 Anthropic ClaudeLog 支援三種傳輸方式：**HTTP**（推薦用於遠端）、**SSE**（Server-Sent Events）、**stdio**（本地行程）。 Anthropic

```
# HTTP 連接 Notion
claude mcp add --transport http notion https://mcp.notion.com/mcp

# stdio 連接 GitHub
claude mcp add github -- npx -y @modelcontextprotocol/server-github

# 帶 OAuth 認證
claude mcp add --transport http api https://api.example.com/mcp \
  --client-id $CLIENT_ID --client-secret $CLIENT_SECRET
```

專案級 MCP 設定寫入根目錄的 `.mcp.json`，版本控制後團隊共享： Anthropic ClaudeLog

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
```

```
      "env": { "GITHUB_TOKEN": "your-token" }
    },
    "notion": {
      "type": "http",
      "url": "https://mcp.notion.com/mcp"
    }
  }
}
```

MCP 工具在 Claude Code 中以 **`mcp__<server-name>__<tool-name>`** 命名（例如 `mcp__github__list_issues`），Anthropic 可在 hooks 的 matcher 和 permissions 的 allow/deny 清單中精確控制。Claude 當 MCP 工具描述超過 context window 的 10% 時，會自動啟用 **Tool Search** 機制按需載入。Anthropic

---

Agents/子代理：獨立上下文的專業分工

子代理運作原理

子代理是擁有**獨立上下文視窗**的特化 Claude 實例。每個子代理有自己的系統提示、工具存取權限和獨立權限。Claude 遇到匹配的任務時自動委派，子代理獨立完成工作後將結果傳回。Anthropic

定義格式

在 `.claude/agents/` 或 `~/.claude/agents/` 中建立 Markdown 檔案：Anthropic +2

```
---
name: security-auditor
description: >-
  資安審計專家。在使用者請求安全審查或修改認證/授權相關程式碼時
  必須主動使用此代理。
tools: Read, Grep, Glob, Bash
model: opus
---

你是一位資深資安工程師。

## 審計流程
1. 掃描依賴漏洞（npm audit / pip-audit）
2. 檢查硬編碼密鑰和敏感資訊
3. 分析認證/授權邏輯
4. 審查輸入驗證與 SQL 注入風險
5. 產出結構化安全報告
```

關鍵欄位： `name` 作為唯一識別、 `description` 決定何時自動委派、 Anthropic `tools` 限制可用工具（省略則繼承父代理所有工具）、 `model` 可指定 `sonnet` / `opus` / `haiku` 。 Anthropic 子代理**無法再產生子代理**（防止無限嵌套），但可以多個子代理**並行執行**。 Anthropic

Agent Teams（2026 年 2 月推出）

Agent Teams 是更高層級的協作機制：多個獨立的 Claude Code 工作階段互相協調、傳訊和分工。 Firecrawl 一個 Lead Agent 分配任務，其他 Teammate 各自獨立工作，能挑戰彼此的發現並匯總結果。 Medium 這適用於大型程式碼庫的並行開發。

Commands：即時可用的斜線指令

兩代格式

舊版（仍支援）： `.claude/commands/` 中的 `.md` 檔案，檔名即指令名。**新版**（推薦）： `.claude/skills/*/SKILL.md` ，統一使用技能格式。 Anthropic Claude

```
---
allowed-tools: Read, Grep, Glob, Bash(git diff:*)
description: 全面的程式碼審查
model: claude-opus-4-6
---

## 已變更的檔案
!`git diff --name-only HEAD~1`

## 詳細差異
!`git diff HEAD~1`

## 審查要點
1. 程式碼品質與可讀性
2. 安全漏洞
3. 效能影響
4. 測試覆蓋率
```

! 反引號語法支援**動態內容注入**，在指令載入時即時執行 shell 指令並嵌入結果。 Claude

五層設定優先級與合併策略

Claude Code 的設定架構採用**五層優先級**，高層覆寫低層： Anthropic

```
┌─ Managed/Enterprise（最高優先級）— IT 管理員部署
├─ Session — CLI --settings 旗標
├─ Project Shared — .claude/settings.json（Git 版控）
├─ Project Local — .claude/settings.local.json（.gitignore）
└─ User Global（最低優先級）— ~/.claude/settings.json
```

Jimmy's Blog

合併規則精確定義：**純量值**高優先級直接勝出、**陣列值**（如 `permissions.allow`）跨層**串接去重**、**物件值**（如 `env`）**深度合併**。 Anthropic Claude 最關鍵的是 **deny 永遠勝出**——任何層級的 deny 規則都無法被其他層級的 allow 覆蓋。 Anthropic Claude Fast

CLAUDE.md 與 SKILL.md 的定位不同：前者是「專案記憶」，**每次對話都載入**（編碼規範、架構決策）； Anthropic Alexop 後者是「專業能力」，**按需載入**（特定領域工作流）。 claude Anthropic

---

結論：一個去中心化但高度結構化的生態系

Claude Code 的可配置架構有三個值得注意的設計決策。第一，**以檔案系統為真相來源**而非中央資料庫，使得版本控制、團隊協作和選擇性安裝成為自然而然的事。第二，**漸進式載入**讓安裝數百個技能不會降低效能——只有元資料永遠在記憶體中。 claude +3 第三，**Agent Skills 開放標準**的野心遠超 Claude 本身——Anthropic 顯然希望它成為「AI 代理能力封裝」的通用格式， Medium Microsoft Community Hub 如同 MCP 成為「AI 代理工具連接」的通用協議。 Unite.AI Anthropic

對於台灣的工程團隊而言，最實用的起步方式是：在專案中建立 `.claude/skills/` 放團隊共享技能、 `.claude/settings.json` 放 hooks 和 permissions（例如 PostToolUse 自動格式化、PreToolUse 阻擋危險操作）， Builder.io 然後用 `/plugin marketplace add anthropics/claude-plugins-official` 探索官方外掛。當團隊規模擴大後，自建內部市集（一個包含 `marketplace.json` 的 Git 倉庫）就能標準化整個組織的 AI 輔助開發工作流。 Claude Builder.io